Hospital Automated Chemical Dosing System
Design Report

CS3243 - IoT Devices and Applications

KR Kanes Rakeshan
230518C

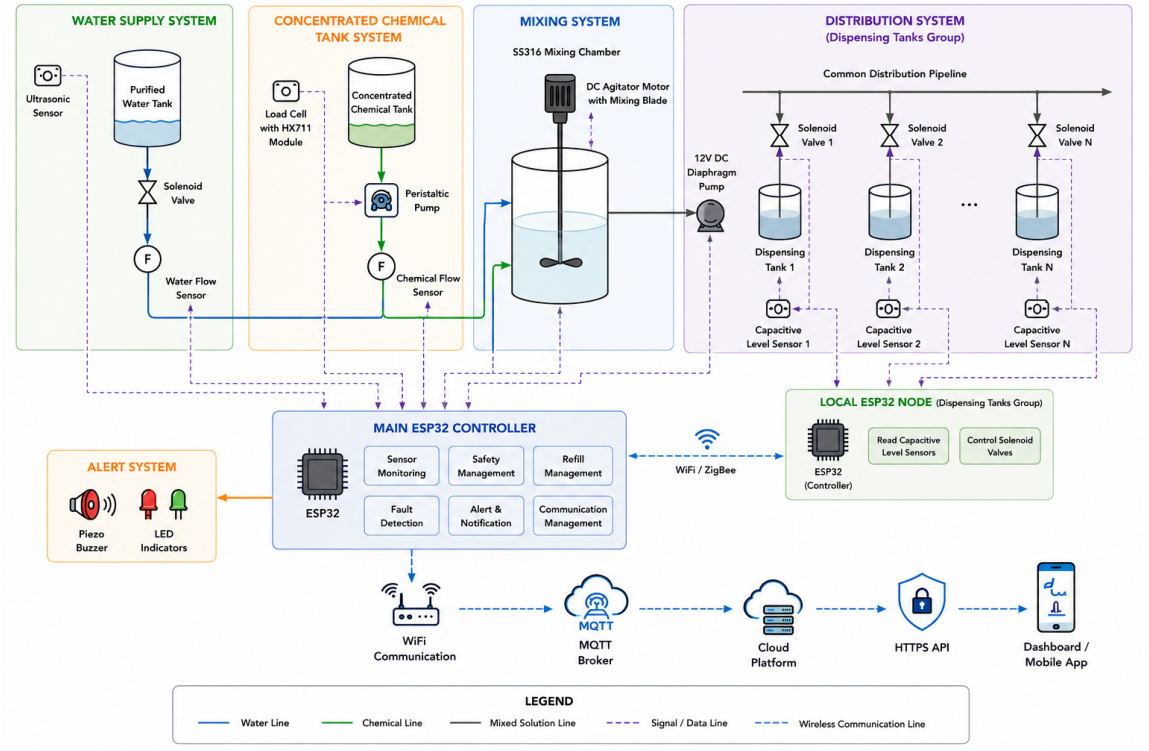
May 29, 2026

Contents

Introduction

Background

Hospitals use specialized disinfectant chemicals to sterilize medical instruments and equipment.



Objectives

- Design a complete IoT-based chemical dispensing solution
- Implement automatic refill control using ESP32
- Monitor water level using non-contact sensing
- Implement fault detection and alarm generation

Proposed IoT System Architecture

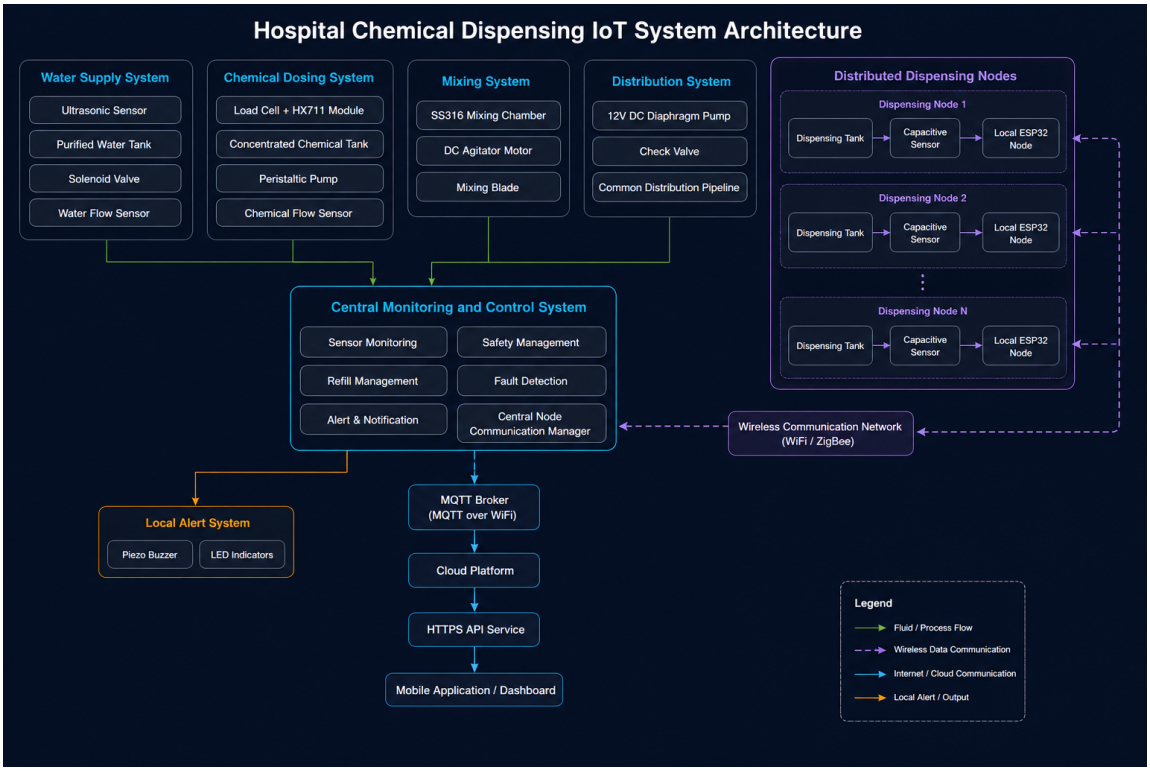


Figure 1: Complete system architecture for the Hospital Automated Chemical Dosing System.

System Overview

The proposed system consists of:

- Purified water tank
- Concentrated chemical supply tank
- SS316 mixing chamber tank with 10L capacity
- Top-mounted agitator system
- Distribution diaphragm pump
- Common dispensing pipeline
- Groups of nearby dispensing tanks (dispensing blocks)
- Local ESP32 nodes placed near each dispensing block for sensor reading and control
- ESP32-based controller

The proposed architecture supports many dispensing blocks placed in different hospital areas. A dispensing block is a small group of tanks that are near each other. Each tank in the block has a solenoid valve and a capacitive level sensor. These sensors and

valves are connected with short wires to a nearby ESP32 node. The local ESP32 reads the sensor signals on its GPIO pins, checks for faults, and sends status messages wirelessly to the central controller.

The peristaltic pump is used only for accurate concentrated chemical dosing into the 10L SS316 stainless steel mixing chamber. Purified water and concentrated chemical enter the mixing chamber, where a top-mounted DC agitator motor with a mixing blade continuously mixes the solution before distribution. After the chemical is uniformly mixed with purified water, a separate 12V DC diaphragm pump transfers the mixed solution from the mixing chamber into the common dispensing pipeline. This pipeline then distributes the prepared solution to the distributed dispensing tank blocks.

Chapter 1
Method to Measure Water Level and Remaining Chemical Levels

1.1 Water Level Measurement and Chemical Estimation

1.1.1 Water Tank Measurement

A waterproof ultrasonic sensor was selected to monitor the purified water tank level since the tank needs regular cleaning and maintenance and the internal sensor installation is not allowed.

The ultrasonic sensor is fixed above the tank, where it detects the water level by calculating the distance to the surface of the liquid. Since the sensor measures the empty space above the water, the actual water level can be determined by subtracting the measured distance from the effective internal height of the tank which is measured when the installation period is completed. We can use the sensor to calibrate for empty tank conditions.

To improve measurement accuracy, the thickness of the tank lid and the wall thickness of the cylindrical tank are considered during calibration. The internal dimensions of the tank are used instead of external dimensions when calculating the actual water capacity.

The tank dimensions, wall thickness, lid thickness, and chemical density values are predetermined during system installation and calibration. These calibrated parameters are stored in the ESP32 controller and are used in subsequent level and volume calculations during system operation.
Let:

- LT = Tank lid thickness
- WT = Tank wall thickness
- H_{ext} = Tank external height
- D_{ext} = Tank external diameter
- d = Measured distance from ultrasonic sensor to water surface

The effective internal tank height is calculated as:

$$H_{int} = H_{ext} - LT - WT \tag{1.1}$$

The effective internal tank radius is:

$$r_{int} = \frac{D_{ext}}{2} - WT \tag{1.2}$$

The total internal tank volume is:

$$V_{tank} = \pi r_{int}^2 H_{int} \tag{1.3}$$

The current water level inside the tank is calculated using:

$$Water\ Level = H_{int} - d \tag{1.4}$$

The remaining water volume is estimated as:

$$V_{water} = \pi r_{int}^2 (Water\ Level) \tag{1.5}$$

This method provides:

- Non-contact operation
- Hygienic measurement
- Low maintenance
- Protection against sensor damage during tank cleaning

1.1.2 Chemical Estimation and Dispensing Control

The system contains two separate liquid systems:

- Concentrated chemical storage tank
- Diluted chemical dispensing tanks

Capacitive level sensors are installed on each dispensing tank in a block to detect low-level refill conditions. These sensors make electrical signals that are read by a nearby ESP32 node using short wires and the node's GPIO inputs. The sensors do not send data directly to the cloud or the central controller.
The concentrated chemical tank uses two monitoring methods:

- YF-S201 flow sensor (hall-effect)
- Load cell weight sensor

The YF-S201 flow sensor continuously measures the amount of concentrated chemical dispensed into the mixing chamber. The local ESP32 controller maintains a cumulative record of the total dispensed chemical volume.
The remaining chemical quantity can be estimated using:

$$Remaining\ Chemical = Initial\ Volume - \sum Dispensed\ Volume \tag{1.6}$$

A load cell placed beneath the concentrated chemical tank continuously measures the tank weight. The measured weight reduction is compared against the expected dispensed chemical amount obtained from the flow sensor.
The relationship between weight and volume is:

$$Mass = Density \times Volume \tag{1.7}$$

The density of the concentrated chemical is assumed to be known beforehand based on manufacturer specifications or laboratory measurements and is stored in the controller during system calibration.

Therefore:

$$Volume = \frac{Mass}{Density} \tag{1.8}$$

This allows the system to estimate the remaining chemical quantity using both flow-based calculation and weight-based verification.
The difference between expected and measured chemical reduction can be calculated as:

$$Error = |Expected\ volume\ reduction - Measured\ volume\ reduction| \tag{1.9}$$

If the error exceeds a predefined threshold volume level, the system detects anomalies such as:

- Pipeline leakage
- Pump malfunction
- Flow sensor failure
- Unexpected chemical loss

The concentrated chemical dosing process is controlled using a peristaltic pump together with a flow sensor. The peristaltic pump is used for accurate dosing into the mixing chamber. Although PWM adjusts the pump speed, it does not guarantee accurate liquid flow because of variations in pressure, resistance in pipe, changes of viscosity and pump wear. Therefore, actual flow measurement is required for accurate dispensing control.

Inside the mixing chamber, a DC agitator motor rotates a mixing blade to maintain consistent mixing. This helps maintain uniform chemical concentration before the solution is supplied to the distribution line.

The mixed chemical then passes through a 12V DC diaphragm pump fixed between the mixing chamber and the main distribution pipeline. The pump helps to move the prepared chemical evenly across multiple dispensing tanks while maintaining a consistent flow throughout the system.

The refill operation only begins after all safety conditions are validated by the local and central esp32 controller. The required pre-conditions to be validated before starting the refill operation are:

- Sufficient water available
- Sufficient concentrated chemical available
- No leakage detected
- No sensor faults detected
- No overflow condition detected

This safety validation process acts as an interlock mechanism to prevent unsafe dispensing conditions.

The refill operation stops automatically once the required target volume has been measured by the flow sensor, mixed in the agitator-equipped mixing chamber, and delivered through the distribution pump and common dispensing pipeline.

Chapter 2
Component Selection

2.1 Selected Components List

- ESP32 DevKit V1 Microcontroller
- Ultrasonic Sensor for Water Tank Level Measurement
- Load Cell with HX711 Module for Concentrated Chemical Tank Monitoring
- YF-S201 Flow Sensor for Flow Measurement
- Capacitive Sensor for Dispensing Tank Level Detection
- Peristaltic Pump for Chemical Dispensing
- SS316 Mixing Chamber Tank (10L)
- DC Agitator Motor with Mixing Blade
- 12V DC Diaphragm Pump for Distribution System
- Normally Closed Solenoid Valve for Flow Control
- Piezo Buzzer for Alarm Indication
- LED Indicators for System Status Display
- MOSFET Driver Module for Pump, Agitator, and Valve Control
- WiFi Communication using ESP32 Built-in Module

2.2 Total Cost Estimate

Component	**Qty**	**Unit Cost (Rs.)**	**Price (Rs.)**
ESP32 DevKit V1 Local Node	1	1,500	1,500
Ultrasonic Sensor (Water Tank Level)	1	1,880	1,880
Load Cell + HX711 Module (Chemical Tank)	1	480	480
Hall-Effect Flow Sensor	1	650	650
Capacitive Sensor (One Tank in the Block)	1	1300	1300
Peristaltic Pump	1	3500	3500
SS316 Mixing Chamber Tank (10L)	1	6000	6000
DC Agitator Motor with Mixing Blade	1	3500	3500
Distribution Diaphragm Pump	1	4500	4500
Normally Closed Solenoid Valve	1	810	810
Piezo Buzzer	1	150	150
LED Indicators (with Resistors)	4	30	120
MOSFET Driver Module	1	350	350
12V DC Power Supply Adapter	1	1800	1800
Wires, Connectors, Tubing (Estimated)	1	1000	1000
		Total	**27,540**

Table 2.1: Estimated Hardware Cost for One Dispensing Block with One Local Node

For each additional tank in the same block, the shared node, pump, mixer, and power parts stay the same. Only the tank-side sensor, valve, and short wiring are added.

Component	**Qty**	**Unit Cost (Rs.)**	**Price (Rs.)**
Capacitive Sensor (One More Tank)	1	1300	1300
Normally Closed Solenoid Valve	1	810	810
Wires, Connectors, and Tubing for One More Tank	1	500	500
		Additional Cost per Tank in the Same Block	**2,610**

Table 2.2: Estimated Additional Hardware Cost for One More Tank in the Same Block

2.3 Controller Comparison

Feature	**ESP32**	**Raspberry Pi 4**	**Arduino UNO**
Processing Capability	Dual-core microcontroller	Single-board computer	Basic microcontroller
Built-in WiFi	Yes	Yes	No
GPIO Pins	Multiple GPIO pins	Multiple GPIO pins	Limited GPIO pins
PWM Support	Yes	Limited native support	Yes
Analog Input Support	Yes	Requires external ADC	Yes
Power Consumption	Low	High	Low
Operating System	Not required	Linux operating system required	Not required
Ease of IoT Integration	Easy	Moderate	Requires external WiFi module
Real-Time Control Capability	Good	Moderate	Good
Approximate Market Price (2026)	RS. 1,100 – 9,000	RS. 8,800 – 125,000	RS. 995 – 3,750
Suitability for This Project	Highly suitable	Overpowered	Limited features
Product Link	`https://www.vseeds.lk/products/esp32-dev-kit-v1`	`https://tronic.lk/product/raspberry-pi-4-model-b-4gb-original-uk`	`https://tronic.lk/product/arduino-uno-original-development-board-with-usb-cable`

Table 2.3: Comparison of ESP32, Raspberry Pi 4, and Arduino UNO

ESP32 was selected because it provides built-in WiFi support, sufficient GPIO pins, low power consumption, and supports real-time monitoring and control at a low cost.

2.4 Water Tank Level Sensor Comparison

Feature	**Ultrasonic Sensor**	**Float Sensor**	**Capacitive Sensor**
Measurement Method	Non-contact distance measurement	Mechanical float detection	Capacitance-based liquid detection
Sensor Contact with Liquid	No	Yes	Usually yes
Maintenance Requirement	Low	Moderate	Moderate
Suitability for Hygienic Environments	Highly suitable	Less suitable	Suitable
Ease of Installation	Easy	Moderate	Moderate
Continuous Monitoring	Yes	Limited	Yes
Corrosion Resistance	High	Moderate	Moderate
Cleaning Compatibility	Excellent	Limited	Moderate
Approximate Market Price (2026)	RS. 1,000 – 2,500	RS. 400 – 2,000	RS. 1,800 – 4,000
Suitability for This Project	Highly suitable	Limited	Moderately suitable
Product Link	`https://tronic.lk/product/jsn-sr04t-waterproof-ultrasonic-distance-measuring-modu`	`https://tronic.lk/product/water-level-sensor-float-switch-right-angle`	`https://tronic.lk/product/non-contact-tank-liquid-water-level-sensor-12-24v`

Table 2.4: Comparison of Water Tank Level Sensors

The ultrasonic sensor was selected because it provides non-contact water level measurement, easy maintenance, and is suitable for hygienic hospital environments.

2.5 Chemical Tank Monitoring Comparison

Feature	**Load Cell Sensor**	**Ultrasonic Sensor**	**Float Sensor**
Measurement Method	Weight measurement	Distance measurement	Mechanical float detection
External Installation	Yes	Yes	No
Sensor Contact with Chemical	No	No	Yes
Leakage Detection	Yes	Limited	No
Maintenance Requirement	Low	Low	Moderate
Suitability for Chemical Tank	Highly suitable	Moderately suitable	Limited
Continuous Monitoring	Yes	Yes	Limited
ESP32 Integration	Easy	Easy	Easy
Approximate Market Price (2026)	RS. 200 – 1,000	RS. 1,000 – 2,500	RS. 400 – 2,000
Suitability for This Project	Highly suitable	Moderately suitable	Limited
Product Link	`https://tronic.lk/product/load-cell-20kg`	`https://tronic.lk/product/jsn-sr04t-waterproof-ultrasonic-distance-measuring-modu`	`https://tronic.lk/product/water-level-sensor-float-switch-right-angle`

Table 2.5: Comparison of Chemical Tank Monitoring Methods

The load cell sensor was selected because it allows external monitoring of the chemical tank and helps detect abnormal chemical loss without placing sensors inside the tank.

2.6 Flow Sensor Comparison

Feature	**Hall-Effect Flow Sensor**	**Turbine Flow Sensor**	**Electromagnetic Flow Sensor**
Measurement Method	Pulse-based magnetic sensing	Rotating turbine measurement	Electromagnetic induction
Ease of Integration with ESP32	Easy	Moderate	Complex
Real-Time Flow Monitoring	Yes	Yes	Yes
Accuracy	Moderate	High	Very high
Maintenance Requirement	Low	Moderate	Low
Cost	Low	Moderate	High
Small System Suitability	Highly suitable	Suitable	Less suitable
Power Consumption	Low	Moderate	High
Approximate Market Price (2026)	RS. 640 – 1,500	RS. 3,000 – 10,000	RS. 20,000 – 80,000
Suitability for This Project	Highly suitable	Moderately suitable	Overpowered
Product Link	`https://tronic.lk/product/yf-s201-water-flow-sensor-1-30lmin-1-75mpa-0-5inch`	`https://lk-tronics.com/product/yf-b10-hall-effect-flow-sensor-water-1-50l-min-turbine-flowmeter/`	`https://fieldinstruments.lk/products/honeywell-smartline-versaflow-mag-4000-electromagnetic-flow-sensor-4805`

Table 2.6: Comparison of Flow Sensors

The Hall-Effect flow sensor was selected because it provides real-time flow measurement, easy ESP32 interfacing, and low implementation cost.

2.7 Dispensing Tank Level Sensor Comparison

Feature	**Capacitive Sensor**	**Float Sensor**	**Optical Sensor**
Detection Method	Capacitance-based sensing	Mechanical float detection	Optical liquid detection
Suitable for Small Tanks	Highly suitable	Moderately suitable	Suitable
Sensor Contact with Liquid	Minimal / external	Yes	Yes
Maintenance Requirement	Low	Moderate	Low
Ease of Installation	Easy	Moderate	Moderate
Reliability	Good	Moderate	Good
Compact Size	Yes	Limited	Yes
Approximate Market Price (2026)	RS. 1,800 – 4,000	RS. 400 – 2,000	RS. 2,000 – 3,000
Suitability for This Project	Highly suitable	Moderately suitable	Suitable
Product Link	`https://tronic.lk/product/non-contact-tank-liquid-water-level-sensor-12-24v`	`https://tronic.lk/product/water-level-sensor-float-switch-right-angle`	`https://lk-tronics.com/product/optical-infrared-water-liquid-level-sensor-with-module/`

Table 2.7: Comparison of Dispensing Tank Level Sensors

The capacitive sensor was selected because it is compact, reliable, and suitable for small dispensing tanks with low maintenance requirements.

2.8 Chemical Dosing Pump Comparison

Feature	Peristaltic Pump	Diaphragm Pump	Submersible Pump
Pumping Method	Tube compression	Diaphragm pressure	Impeller-based pumping
Chemical Contact with Pump	No direct contact	Direct contact	Direct contact
Dispensing Accuracy	High	Moderate	Low
Speed Control	Easy	Moderate	Limited
Suitability for Chemical Dispensing	Highly suitable	Suitable	Limited
Maintenance Requirement	Low	Moderate	Moderate
Contamination Risk	Low	Moderate	High
Ease of Integration	Easy	Moderate	Easy
Approximate Market Price (2026)	RS. 2,500 – 4,500	RS. 1,500 – 8,000	RS. 850 – 4,900
Suitability for This Project	Highly suitable	Moderately suitable	Limited
Product Link	`https://www.daraz.lk/tag/12v-peristaltic-pump/`	`https://tronic.lk/product/diaphragm-water-pump-12v-7lmin-hy-5200`	`https://lk-tronics.com/product-tag/water-pump/`

Table 2.8: Comparison of Chemical Dosing Pump Types

The peristaltic pump was selected because it provides accurate dispensing, low contamination risk, and supports controlled chemical flow.

2.9 Mixing Method Comparison

Feature	**Mechanical Agitator**	**Magnetic Stirrer**	**Natural Turbulent Mixing**
Mixing Performance	High and uniform mixing using a rotating blade	Good for small laboratory volumes	Low and depends on inlet flow turbulence
Suitable Volume Range	Suitable for 10L and larger small industrial tanks	Mostly suitable for small beakers and laboratory containers	Suitable only for simple low-accuracy mixing
Continuous Operation	Suitable for continuous operation with motor speed control	Possible, but limited by stirrer capacity and heating plate design	No active control over mixing continuity
Maintenance Requirement	Moderate due to motor, shaft, and blade cleaning	Low to moderate	Low
Industrial Suitability	Highly suitable for small process tanks	Limited for industrial tank mixing	Limited for controlled chemical concentration
Cost	Moderate	High	Minimal
Approximate Market Price (2026)	RS. 3,000 – 10,000	RS. 10,000 – 20,000	Minimal
Suitability for This Project	Highly suitable	Moderately suitable	Limited
Product Link	`https://www.daraz.lk/tag/dc-geared-motor/`	`https://www.daraz.lk/tag/magnetic-stirrer/`	N/A

Table 2.9: Comparison of Mixing Methods

The mechanical agitator system was selected because it provides reliable continuous mixing, supports larger liquid volumes, and ensures uniform chemical concentration before distribution to dispensing tanks.

2.10 Distribution Pump Comparison

Feature	Diaphragm Pump	Centrifugal Pump	Peristaltic Pump
Flow Capability	Moderate to high flow for small distribution systems	High flow but depends strongly on priming and installation	Low to moderate flow, mainly suitable for dosing
Pressure Stability	Good pressure output with self-priming operation	Moderate pressure stability, affected by head and pipeline load	Pulsed flow, stable for small metered dosing only
Pipeline Distribution Suitability	Highly suitable for common pipeline distribution to multiple tanks	Suitable for simple water transfer, but less suitable for controlled small pipelines	Limited because the flow rate is low for distribution use
Maintenance Requirement	Moderate	Low to moderate	Low, but tube replacement is required
Cost	Moderate	Low to moderate	Moderate
Approximate Market Price (2026)	RS. 1,500 – 8,000	RS. 850 – 4,900	RS. 2,500 – 4,500
Suitability for This Project	Highly suitable	Moderately suitable	Limited for distribution; suitable for dosing only
Product Link	`https://tronic.lk/product/diaphragm-water-pump-12v-7lmin-hy-5200`	`https://lk-tronics.com/product-tag/water-pump/`	`https://www.daraz.lk/tag/12v-peristaltic-pump/`

Table 2.10: Comparison of Distribution Pump Types

The diaphragm pump was selected for the distribution system because it provides stable pressure, supports multiple dispensing tanks, and is suitable for continuous liquid transfer through common pipelines.

2.11 Valve Comparison

Feature	**Solenoid Valve**	**Motorized Ball Valve**	**Pinch Valve**
Operating Method	Electromagnetic control	Motor-driven rotation	Tube compression
Response Speed	Fast	Moderate	Fast
ESP32 Control	Easy	Moderate	Moderate
Leakage Protection	Good	Excellent	Excellent
Power Consumption	Moderate	Low	Moderate
Maintenance Requirement	Low	Moderate	Low
Suitability for Chemical Systems	Suitable	Highly suitable	Suitable
Power Failure Safety	Yes	Depends on design	Yes
Approximate Market Price (2026)	RS. 800 – 16,000	RS. 15,000 – 30,000	RS. 10,000 – 25,000
Suitability for This Project	Highly suitable	Moderately suitable	Overpowered
Product Link	`https://tronic.lk/product/electric-solenoid-valve-12-12vdc-nc-plastic-water`	`https://lk-tronics.com/product/24v-dc-2-way-motorized-ball-valve/`	`https://www.daraz.lk/products/pinch-valve-industrial-i318742219.html`

Table 2.11: Comparison of Valve Types

The normally closed solenoid valve was selected because it provides fast automatic flow control, easy ESP32 interfacing, and automatic shutoff during power failure.

Chapter 3
Failure Detection and Fail-Safe Mechanisms

3.1 Monitoring and Detection Methods

The proposed automated chemical dispensing system uses both sensor-based and computational methods for monitoring system operation and detecting abnormal conditions.

3.1.1 Sensor-Based Monitoring Methods

- Ultrasonic sensor for purified water tank level monitoring
- Capacitive level sensors mounted on each dispensing tank block to detect low-level refill conditions through a local ESP32 node
- YF-S201 flow sensor for measuring liquid flow rate and dispensed volume
- Load cell sensor with HX711 module for monitoring concentrated chemical quantity locally

3.1.2 Computational and Algorithmic Methods

- Remaining chemical estimation using cumulative flow measurements
- Detection of abnormal flow conditions using flow pulse monitoring
- Refill timeout monitoring for detecting pump or valve failures
- Agitator operation monitoring during the mixing stage
- Leakage detection by comparing expected and measured chemical reduction
- Sequential refill scheduling algorithm for managing multiple dispensing tanks

3.2 Potential Failures and Detection Methods

Potential Failure	**Detection Method**	**Fail-Safe Method**
Empty Water Tank	Ultrasonic sensor detects low water level	Stop refill operation and generate alarm
Empty Chemical Tank	Load cell detects low remaining weight	Stop pump operation and notify operator
Pump Failure	No flow pulses detected during refill	Stop system and activate alarm
Distribution Pump Failure	Low delivery flow or refill timeout after mixing	Stop distribution pump, close valves, and activate alarm
Agitator Failure	No motor operation detected during mixing stage	Stop refill operation and prevent distribution until mixing is restored
Blocked Valve or Pipeline	Low or no flow detected by flow sensor	Close valve and stop pump
Pipeline Leakage	Mismatch between expected and measured chemical quantity	Stop dispensing and generate warning
Sensor Failure	Invalid or unstable sensor readings	Switch system to safe shutdown mode
Overflow Condition	Refill operation exceeds expected refill time	Automatically stop refill process
Communication Failure	No response from sensor modules or controller	Generate maintenance warning and stop automatic operation
Power Failure	Loss of power supply detected	Normally closed valve automatically stops liquid flow

Table 3.1: Potential Failures, Detection Methods, and Fail-Safe Mechanisms

3.3 Fail-Safe Strategy

The system is designed with multiple fail-safe mechanisms to reduce operational risks in hospital environments. Automatic shutdown procedures are used during abnormal conditions such as empty tanks, pump failures, leakage detection, or communication failures. Audible alarms and warning notifications are also generated to alert maintenance personnel.

Normally closed solenoid valves are used to automatically stop liquid flow during power failures, reducing the risk of leakage or uncontrolled dispensing. The use of both sensor-based monitoring and computational estimation improves overall system reliability and fault detection capability.

Chapter 4
Data Collection and Communication System

4.1 Data Collection from Dispensing Tanks

The dispensing tanks are spread across different areas of the hospital and need regular checks for refills and faults. Each dispensing block is a small group of tanks placed close to one another. Each tank has a solenoid valve and a capacitive level sensor. The sensors and valves are wired to a nearby ESP32 node with short cables. The ESP32 reads the sensor signals and handles safety checks. The sensor signals are not sent directly to the cloud or to a central controller.
Sensor data collected from the dispensing tanks includes:

- Tank level
- Flow pulse counts
- Fault and overflow alerts

Local ESP32 nodes collect and pre-process sensor readings and forward compact telemetry to the central controller over wireless. The central controller manages refill scheduling and alarms; the cloud (MQTT) stores telemetry for monitoring and remote access only.
This architecture supports:

- Distributed dispensing tank monitoring across multiple hospital locations
- Centralized refill management with local node-based validation
- Real-time alarm handling at the local node and central controller
- Cloud-based monitoring and logging
- Remote maintenance access through secure web and mobile clients

4.2 Communication Method Comparison

WiFi and ZigBee were compared for range, power, scalability, complexity, and ESP32 support.

For areas where WiFi is unreliable or unavailable (for example, basements or remote wings), LoRa/LoRaWAN is an alternative. LoRa gives long range and low power use, but it has low data rates and needs a gateway and network server. LoRa is good for occasional telemetry such as tank level updates, but not for high-rate flow data or real-time control. Using LoRa requires extra hardware (LoRa transceivers or gateway) and planning gateway placement to cover the blocks.

Feature	**WiFi**	**ZigBee**
Communication Range	Moderate in building coverage	Moderate to High
Power Consumption	High	Low
Data Transmission Speed	High	Lower than WiFi
Ease of Implementation	Easy	Moderate
Built-in ESP32 Support	Yes	No
Network Scalability	Moderate	High
Mesh Networking Support	Limited	Yes
Hospital Infrastructure Suitability	Strong fit with existing hospital WiFi and MQTT integration	Suitable for large meshes, but requires gateways and extra planning
Approximate Hardware Cost	Low	Moderate
Hardware Module Link	Available in ESP32	`https://tronic.lk/product/xbee-zigbee-s2c`

Table 4.1: Comparison of WiFi and ZigBee Communication Methods

4.3 Selected Communication Method

WiFi communication was selected as the primary communication method for the proposed prototype because the ESP32 controller provides built-in WiFi capability without requiring additional communication modules.

WiFi supports wireless communication between the distributed local ESP32 nodes and the central controller through the hospital wireless network infrastructure. This allows real-time monitoring of dispensing tank levels, refill requests, flow conditions, and system alarms without direct long-distance sensor wiring.

Compared with ZigBee, WiFi provides:

- Higher data transmission speed
- Easier Internet connectivity
- Simpler ESP32 integration
- Lower implementation complexity for the prototype system

Although ZigBee provides lower power consumption and better mesh-network capability for very large distributed systems, it requires additional hardware modules and more complex network configuration. It is therefore more suitable for large mesh deployments than for a low-complexity prototype.

For IoT monitoring functionality, MQTT protocol is used over the WiFi network to transmit system telemetry data between the central ESP32 controller and the cloud platform. MQTT provides lightweight and reliable communication suitable for real-time IoT monitoring systems.

The communication architecture of the proposed system is:

Dispensing Tank Sensors → Local ESP32 Node → Wireless Communication Network → Central ESP32 Controller → MQTT Broker → Cloud Platform → HTTPS API → Dashboard or Mobile Application

HTTPS communication is used between the cloud platform and the dashboard or mobile application to provide secure remote access for hospital operators and maintenance personnel.

WiFi with MQTT was selected because the proposed system operates inside a hospital building with existing WiFi infrastructure and requires real-time cloud monitoring capability.

How dispensing tank data is collected and communicated:

- Each dispensing block is a small group of tanks monitored by a nearby ESP32 node. The sensors and valves are connected to that node with short wires, not long runs back to the central controller.
- The local ESP32 node reads the capacitive level sensor, executes local safety checks, and publishes telemetry to an MQTT broker over WiFi.
- The cloud platform subscribes to MQTT topics to aggregate telemetry. Mobile and web applications access aggregated data from the cloud platform over HTTPS.
- For hospital-wide deployment, additional WiFi access points can extend coverage. ZigBee can be considered for very large mesh networks, but it adds hardware and configuration complexity.

Chapter 5
Complete System
Schematic Block Diagram

Complete schematic block diagram of all system components.

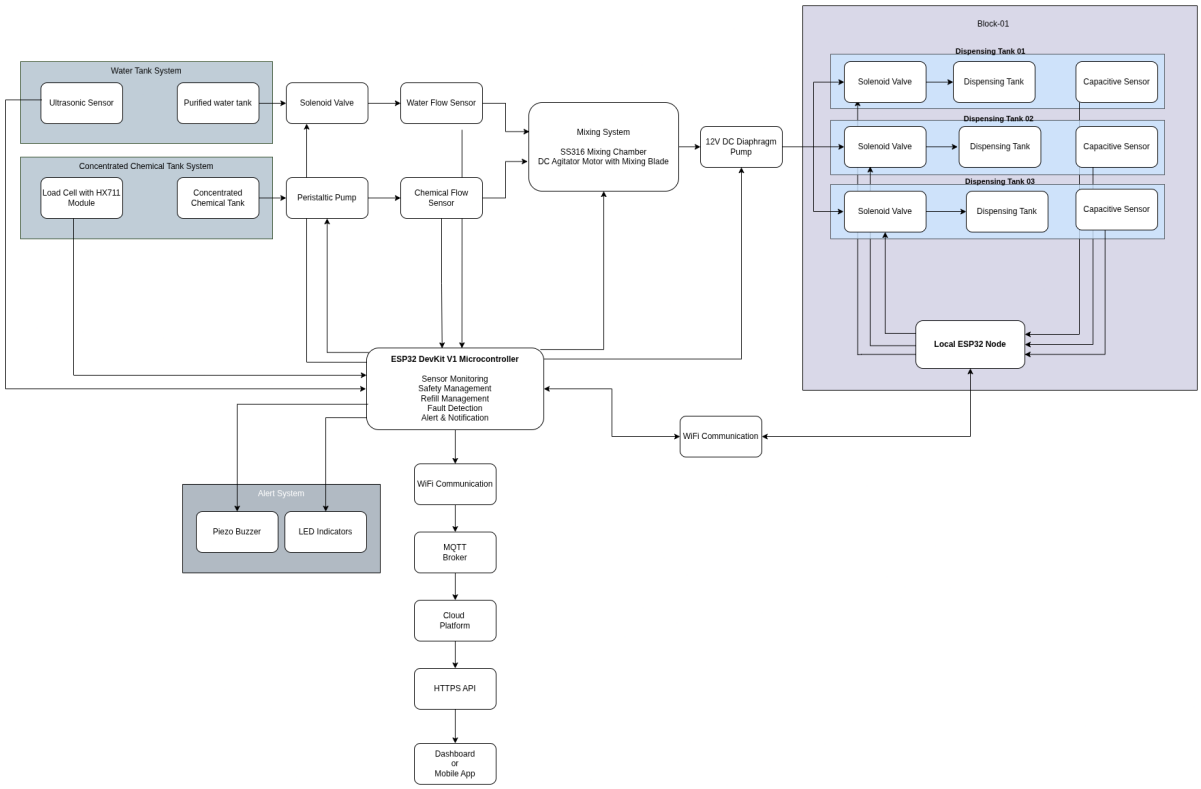
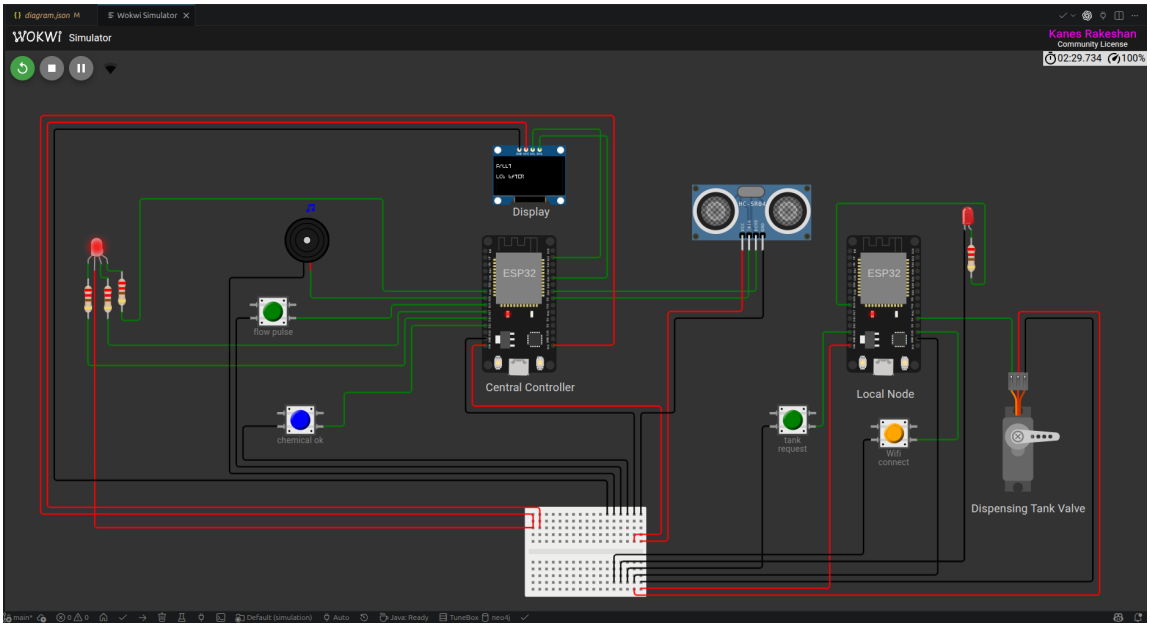
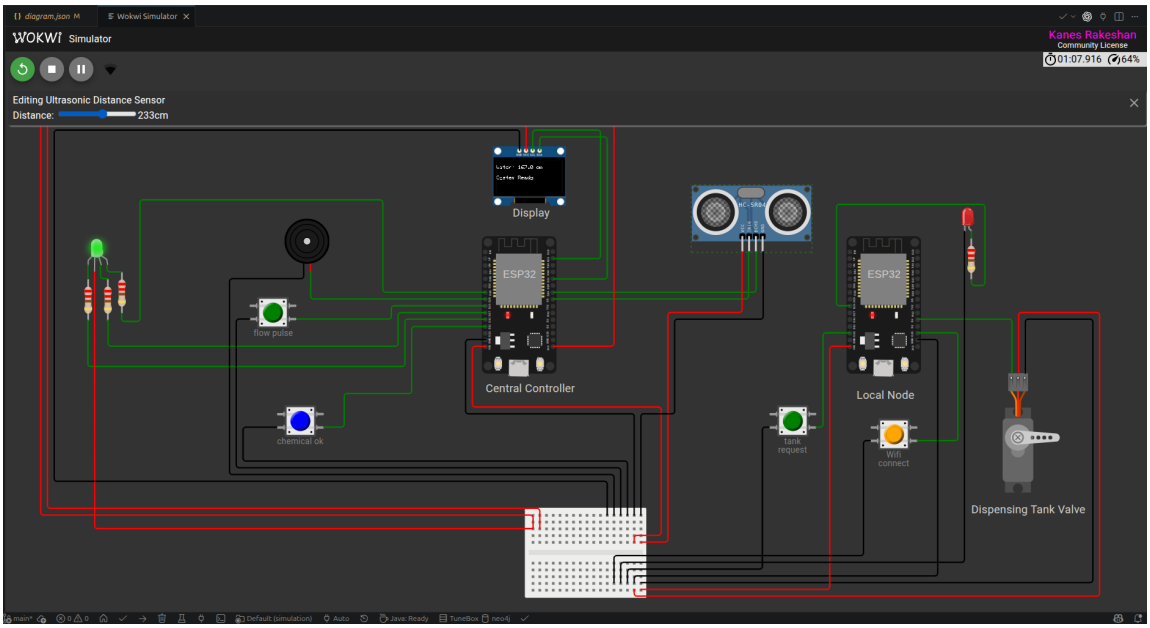


Figure 5.1: Complete system block diagram for the Hospital Automated Chemical Dosing System.

Chapter 6
Controller and Sensor Connection Schematic





Signal / Device	Controller (Central) Pin	Purpose
Ultrasonic TRIG	D5	Trigger pulse for distance measurement
Ultrasonic ECHO	D18	Echo input (distance)
OLED (SDA / SCL)	D21 / D22	I2C display (0x3C)
RGB LED (R/G/B)	D14 / D27 / D33	System status indicators
Buzzer	D25	Audible alarm/notification
Flow pulse input	D26	Flow pulse / YF-S201 input (simulated)
Chemical OK (push)	D12	Chemical / load-cell status (simulated)
Serial	TX0 / RX0	Debug / serial monitor

Table 6.1: Controller pin summary (matches simulation/diagram.json and controller code).

Signal / Device	Local Node Pin	Purpose
Tank request button	D13	Manual refill request (active-low)
Servo (valve emulation)	D4	SG90 servo PWM (simulates solenoid)
Wifi connect button	D15	Request Wifi connect
Blink LED	D26	Status blink LED

Table 6.2: Local node pin summary (matches simulation wiring; local code pins shown where relevant).

Chapter 7
Operational Flow Chart for IoT Components

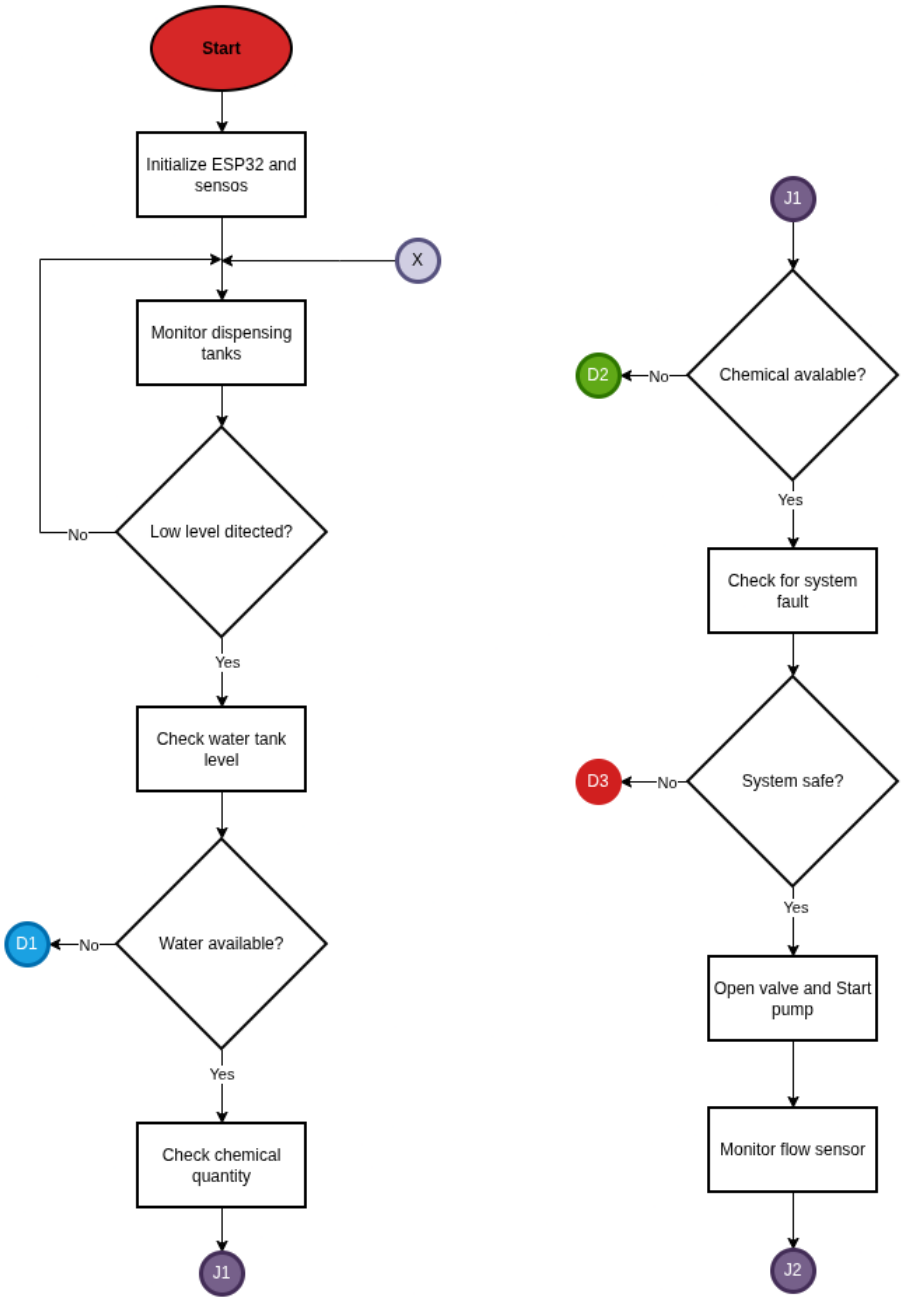


Figure 7.1: *

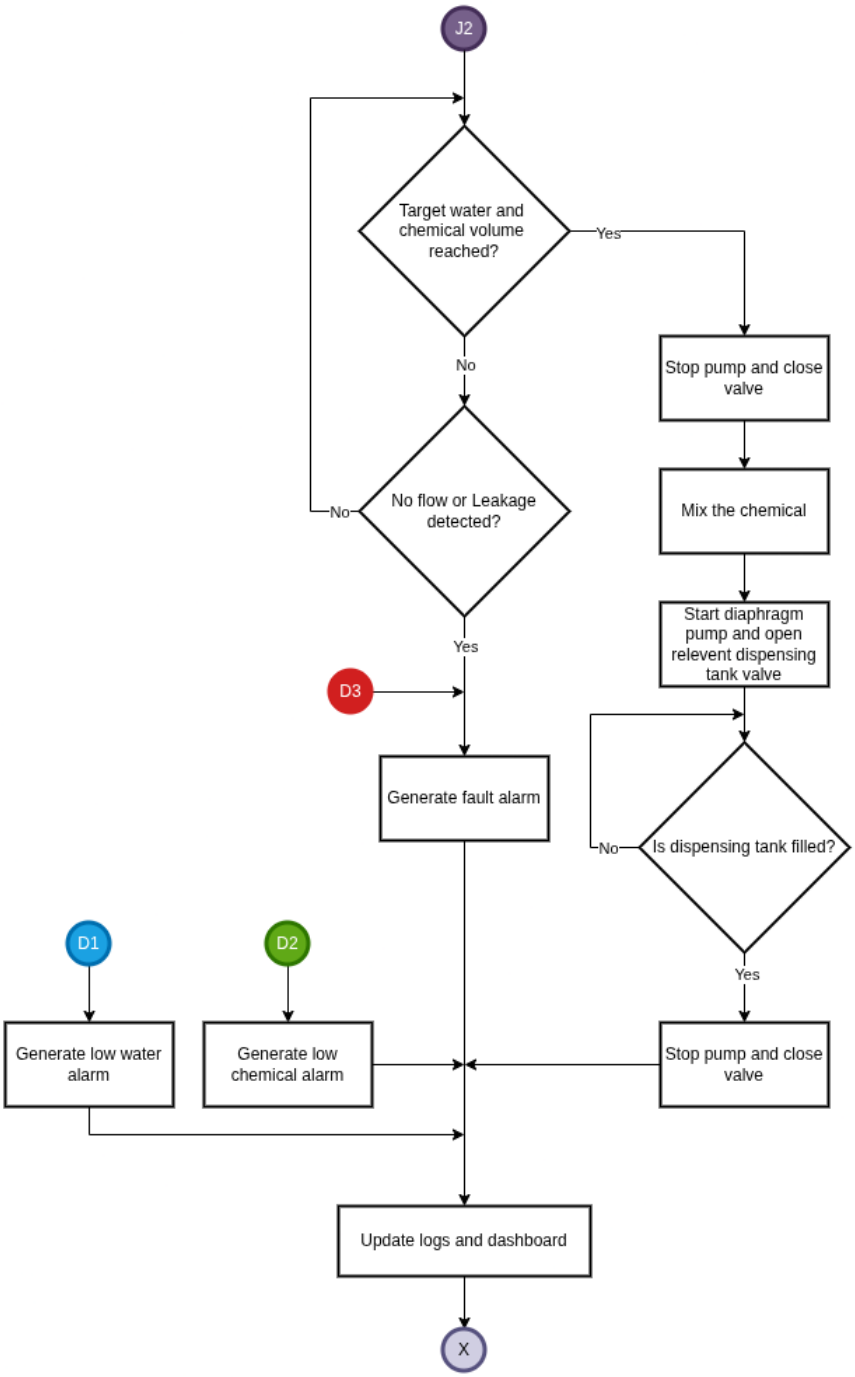

J2
Target water and chemical volume reached?
Yes
No
Stop pump and close valve
No flow or Leakage detected?
No
Mix the chemical
Start diaphragm pump and open relevent dispensing tank valve
Yes
D3
Generate fault alarm
Is dispensing tank filled?
No
D1
D2
Yes
Generate low water alarm
Generate low chemical alarm
Stop pump and close valve
Update logs and dashboard
X

Figure 7.2: *

Chapter 8
Simulation of the prototype system in WokWi

8.1 Simulation

The Wokwi simulation implements a prototype demonstrating a single dispensing node interacting with a central controller. The simulation models peripheral modules and simplifies hardware behavior to validate software logic and communication flows.
Simulated components:

- ESP32 DevKit V1 — main controller board used for local node and central controller firmware.
- HC-SR04 ultrasonic sensor — emulates tank level measurement.
- SSD1306 OLED (I2C, address 0x3C) — status display.
- SG90 servo — emulates solenoid valve actuation (used only because Wokwi does not simulate solenoids).
- RGB LED with series resistors — visual status indicators.
- Piezo buzzer — audible alerts.
- Push-buttons — emulate tank refill request, flow pulse input, and chemical/load-cell status.
- Breadboard — power distribution and common ground for components in the diagram.

Simulation notes:

- The servo is a substitute for the normally-closed solenoid valve used in the physical system; it is included only for visualization in Wokwi.
- Load cell behavior is approximated by a push-button in the simulation. In hardware this would be provided by a load cell and HX711 amplifier.
- Flow pulse generation is emulated using a button; a YF-S201 flow sensor would provide pulse signals in the real system.
- The simulation demonstrates control and monitoring logic for a single node; the production architecture supports multiple distributed nodes.
- The simulation is intended for functional verification of software and signal flows and does not model fluid dynamics or mixing physics.

8.2 Source Code

8.2.1 diagram.json file

```
{
  "version": 1,
  "author": "Kanes Rakeshan",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-breadboard-mini",
      "id": "bb1",
      "top": 430.6,
      "left": 237.6,
      "attrs": { "color": "#eeefed" }
    },
    {
      "type": "wokwi-esp32-devkit-v1",
      "id": "centralNodeEsp",
      "top": 43.1,
      "left": 177.4,
      "attrs": {
        "firmware": ".pio/build/controller/firmware.bin",
        "elf": ".pio/build/controller/firmware.elf",
        "label": "Central Controller"
      }
    },
    {
      "type": "wokwi-esp32-devkit-v1",
      "id": "localNodeEsp",
      "top": 43.1,
      "left": 695.8,
      "attrs": {
        "firmware": ".pio/build/local_node/firmware.bin",
        "elf": ".pio/build/local_node/firmware.elf",
        "label": "Local Node"
      }
    },
    { "type": "wokwi-hc-sr04", "id": "ultrasonic", "top": -27.3,
      "left": 475.9, "attrs": {} },
    {
      "type": "wokwi-pushbutton",
      "id": "tankRequestBtn",
      "top": 284.6,
      "left": 585.6,
      "attrs": { "color": "green", "xray": "1", "label": "tank
        request" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "flowPulseBtn",
      "top": 131,
      "left": -153.6,
      "attrs": { "color": "green", "xray": "1", "label": "flow pulse" }
    },
    {
      "type": "wokwi-pushbutton",
```

```
      "id": "chemicalOkBtn",
      "top": 284.6,
      "left": -115.2,
      "attrs": { "color": "blue", "xray": "1", "label": "chemical ok" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "heartbeatBtn",
      "top": 303.8,
      "left": 729.6,
      "attrs": { "color": "orange", "xray": "1", "label": "Wifi
         connect" }
    },
    {
      "type": "wokwi-buzzer",
      "id": "buzzer",
      "top": 12,
      "left": -103.8,
      "attrs": { "volume": "0.1" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 129.6,
      "left": -413.35,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": 120,
      "left": -365.35,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": 129.6,
      "left": -384.55,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "board-ssd1306",
      "id": "oled1",
      "top": -83.26,
      "left": 192.23,
      "attrs": { "i2cAddress": "0x3c" }
    },
    {
      "type": "wokwi-servo",
      "id": "servo1",
      "top": 265,
      "left": 855.4,
      "rotate": 90,
      "attrs": { "hornColor": "#ccc", "label": "solenoid Valve" }
```

```
    },
    { "type": "wokwi-rgb-led", "id": "rgb1", "top": 42.4, "left":
       -392.5, "attrs": {} },
    {
      "type": "wokwi-led",
      "id": "led_blink",
      "top": -3.6,
      "left": 848.6,
      "attrs": { "color": "red", "flip": "" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r4",
      "top": 72,
      "left": 844.25,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-text",
      "id": "text1",
      "top": 249.6,
      "left": 182.4,
      "attrs": { "text": "Central Controller" }
    },
    {
      "type": "wokwi-text",
      "id": "text2",
      "top": 259.2,
      "left": 710.4,
      "attrs": { "text": "Local Node" }
    },
    {
      "type": "wokwi-text",
      "id": "text3",
      "top": 422.4,
      "left": 873.6,
      "attrs": { "text": "Dispensing Tank Valve" }
    },
    {
      "type": "wokwi-text",
      "id": "text4",
      "top": 0,
      "left": 220.8,
      "attrs": { "text": "Display" }
    }
  ],
  "connections": [
    [ "localNodeEsp:TX0", "$serialMonitor:RX", "", [] ],
    [ "localNodeEsp:RX0", "$serialMonitor:TX", "", [] ],
    [ "tankRequestBtn:2.r", "localNodeEsp:D13", "green", [ "h9.8",
       "v-127.5" ] ],
    [ "flowPulseBtn:2.r", "centralNodeEsp:D26", "green", [ "h134.6",
       "v-22" ] ],
    [ "chemicalOkBtn:2.r", "centralNodeEsp:D12", "green", [ "h29",
       "v-47.8", "h96", "v-98.4" ] ],
    [ "heartbeatBtn:2.r", "localNodeEsp:D15", "green", [ "h57.8",
       "v-153.2" ] ],
```

```
    [ "buzzer:2", "centralNodeEsp:D25", "green", [ "v19.2", "h-0.4",
       "v35.3" ] ],
    [ "ultrasonic:TRIG", "centralNodeEsp:D5", "green", [ "h-0.9",
       "v36.8" ] ],
    [ "ultrasonic:GND", "centralNodeEsp:GND", "black", [] ],
    [ "tankRequestBtn:1.l", "localNodeEsp:GND", "black", [] ],
    [ "heartbeatBtn:1.l", "localNodeEsp:GND", "black", [] ],
    [ "flowPulseBtn:1.l", "centralNodeEsp:GND", "black", [] ],
    [ "chemicalOkBtn:1.l", "centralNodeEsp:GND", "black", [] ],
    [ "buzzer:1", "centralNodeEsp:GND", "black", [] ],
    [ "centralNodeEsp:D18", "ultrasonic:ECHO", "green", [ "h0" ] ],
    [ "oled1:SDA", "centralNodeEsp:D21", "green", [ "v-19.2",
       "h96.07", "v198.9" ] ],
    [ "oled1:SCL", "centralNodeEsp:D22", "green", [ "v-28.8", "h96.3",
       "v179.6" ] ],
    [ "servo1:PWM", "localNodeEsp:D4", "green", [ "h-0.2", "v-76.9" ]
       ],
    [ "led_blink:A", "r4:1", "green", [ "v0" ] ],
    [ "r4:2", "localNodeEsp:D26", "green", [ "v8.4", "h19.2",
       "v-115.2", "h-211.2", "v144.1" ] ],
    [ "rgb1:R", "r1:1", "green", [ "v0" ] ],
    [ "rgb1:G", "r3:1", "green", [ "v0" ] ],
    [ "rgb1:B", "r2:1", "green", [ "v0" ] ],
    [ "r1:2", "centralNodeEsp:D14", "green", [ "v66", "h451.2",
       "v-70.4" ] ],
    [ "r3:2", "centralNodeEsp:D27", "green", [ "v37.2", "h412.8",
       "v-50.8" ] ],
    [
      "centralNodeEsp:D33",
      "r2:2",
      "green",
      [ "h-146.6", "v-131.3", "h-345.6", "v172.8", "h-28.8" ]
    ],
    [ "servo1:GND", "bb1:17b.i", "black", [ "v-76.8", "h96", "v374.4"
       ] ],
    [ "tankRequestBtn:2.l", "bb1:15b.i", "black", [ "h-9.6", "v201.8",
       "h-192" ] ],
    [ "bb1:17t.e", "ultrasonic:VCC", "red", [ "h38.4", "v-326.4",
       "h105.6" ] ],
    [
      "centralNodeEsp:VIN",
      "bb1:16t.e",
      "red",
      [ "h-19.2", "v86.4", "h268.8", "v182.4", "h-38.4" ]
    ],
    [ "localNodeEsp:GND.1", "bb1:16b.i", "black", [ "h28.5", "v335.9",
       "h-432" ] ],
    [ "rgb1:COM", "bb1:1t.c", "red", [ "h0.1", "v364.4", "h624" ] ],
    [ "oled1:VCC", "bb1:2t.c", "red", [ "v-38.4", "h-681.45",
       "v547.2", "h700.8" ] ],
    [
      "centralNodeEsp:3V3",
      "bb1:1t.c",
      "red",
      [ "h86.1", "v-326.4", "h-816", "v566.4", "h700.8" ]
    ],
    [ "localNodeEsp:VIN", "bb1:17b.j", "red", [ "h-28.8", "v345.6" ] ],
```

```
    [ "servo1:V+", "bb1:16b.j", "red", [ "h-0.1", "v-86.3", "h115.2",
       "v403.1", "h-662.4" ] ],
    [ "ultrasonic:GND", "bb1:17t.a", "black", [ "v96", "h-126",
       "v134.4", "h-48" ] ],
    [ "centralNodeEsp:GND.2", "bb1:16t.a", "black", [ "h-28.8",
       "v105.5", "h240" ] ],
    [ "chemicalOkBtn:2.l", "bb1:15t.a", "black", [ "h-48", "v48.2",
       "h547.2" ] ],
    [ "flowPulseBtn:2.l", "bb1:14t.a", "black", [ "h-19.2", "v211.4",
       "h547.2" ] ],
    [ "buzzer:1", "bb1:13t.a", "black", [ "v9.6", "h-105.6", "v278.4",
       "h547.2" ] ],
    [ "oled1:GND", "bb1:12t.a", "black", [ "v-28.8", "h-662.4",
       "v499.2", "h787.2" ] ],
    [ "heartbeatBtn:2.l", "bb1:13b.i", "black", [ "h-9.6", "v163.4",
       "h-355.2" ] ],
    [ "led_blink:C", "bb1:14b.i", "black", [ "v470.4", "h-489.2" ] ]
  ],
  "dependencies": {}
}
```

Listing 8.1: Wokwi project diagram json

8.2.2 Controller Source Code

```
#include <Arduino.h>
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <ESPmDNS.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// Use Wokwi default virtual WiFi so both devices join the same network
const char *AP_SSID = "Wokwi-GUEST";
const char *AP_PASSWORD = ""; // open network on Wokwi
WebServer server(80);

#define FLOW_PULSE_PIN 26
#define CHEMICAL_OK_PIN 12
#define RGB_RED_PIN 14
#define RGB_GREEN_PIN 27
#define RGB_BLUE_PIN 33
#define BUZZER_PIN 25
#define TRIG_PIN 5
#define ECHO_PIN 18

volatile int flowPulseCount = 0;
bool refillSessionActive = false;
bool lowWaterAlertActive = false;
bool faultLatched = false;
unsigned long lastFlowPulseTime = 0;
unsigned long lastHeartbeatTime = 0;
```

```
const unsigned long HEARTBEAT_LED_MS = 200;
const int TARGET_PULSES = 10;
const unsigned long FLOW_TIMEOUT = 5000;
const float TANK_HEIGHT_CM = 400.0;
const float LOW_WATER_THRESHOLD_CM = 20.0;
float simulatedDistanceCM = -1.0;
bool rgbCommonAnode = true;

void IRAM_ATTR flowISR()
{
  flowPulseCount++;
  lastFlowPulseTime = millis();
}

static void setRGBColor(bool r, bool g, bool b)
{
  if (rgbCommonAnode)
  {
    digitalWrite(RGB_RED_PIN, r ? LOW : HIGH);
    digitalWrite(RGB_GREEN_PIN, g ? LOW : HIGH);
    digitalWrite(RGB_BLUE_PIN, b ? LOW : HIGH);
  }
  else
  {
    digitalWrite(RGB_RED_PIN, r ? HIGH : LOW);
    digitalWrite(RGB_GREEN_PIN, g ? HIGH : LOW);
    digitalWrite(RGB_BLUE_PIN, b ? HIGH : LOW);
  }
}

static void displayMessage(const String &l1, const String &l2)
{
  display.clearDisplay();
  display.setCursor(0, 10);
  display.println(l1);
  display.setCursor(0, 30);
  display.println(l2);
  display.display();
}

static float getDistanceCM()
{
  if (simulatedDistanceCM >= 0.0)
  {
    return simulatedDistanceCM;
  }

  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  if (duration <= 0)
  {
    return TANK_HEIGHT_CM;
  }
```

```
91
92    return duration * 0.034 / 2.0;
93  }
94
95  static float getWaterLevelCM()
96  {
97    float distance = getDistanceCM();
98    float waterLevel = TANK_HEIGHT_CM - distance;
99    if (waterLevel < 0)
100   {
101     waterLevel = 0;
102   }
103   return waterLevel;
104 }
105
106 static void faultState()
107 {
108   setRGBColor(true, false, false);
109   for (int i = 0; i < 5; i++)
110   {
111     digitalWrite(BUZZER_PIN, HIGH);
112     delay(250);
113     digitalWrite(BUZZER_PIN, LOW);
114     delay(250);
115   }
116 }
117
118 static void startRefillSession()
119 {
120   refillSessionActive = true;
121   flowPulseCount = 0;
122   lastFlowPulseTime = millis();
123   faultLatched = false;
124   displayMessage("REFILL REQUEST", "Approved");
125 }
126
127 static void stopRefillSession()
128 {
129   refillSessionActive = false;
130   setRGBColor(false, true, false);
131 }
132
133 static void handleRequest()
134 {
135   float waterLevel = getWaterLevelCM();
136   bool chemicalOk = digitalRead(CHEMICAL_OK_PIN) == LOW;
137
138   if (refillSessionActive)
139   {
140     server.send(409, "text/plain", "BUSY");
141     return;
142   }
143
144   if (waterLevel < LOW_WATER_THRESHOLD_CM)
145   {
146     displayMessage("FAULT", "LOW WATER");
147     faultLatched = true;
148     faultState();
```

```
149       server.send(200, "text/plain", "DENY");
150       return;
151     }
152
153     if (!chemicalOk)
154     {
155       displayMessage("FAULT", "CHEMICAL EMPTY");
156       faultLatched = true;
157       faultState();
158       server.send(200, "text/plain", "DENY");
159       return;
160     }
161
162     startRefillSession();
163     server.send(200, "text/plain", "APPROVE");
164   }
165
166   static void handleControl()
167   {
168     if (faultLatched)
169     {
170       server.send(200, "text/plain", "FAULT");
171       return;
172     }
173
174     if (!refillSessionActive)
175     {
176       server.send(200, "text/plain", "CLOSE");
177       return;
178     }
179
180     if (flowPulseCount >= TARGET_PULSES)
181     {
182       stopRefillSession();
183       displayMessage("REFILL DONE", "Target Reached");
184       server.send(200, "text/plain", "CLOSE");
185       return;
186     }
187
188     if (millis() - lastFlowPulseTime > FLOW_TIMEOUT)
189     {
190       displayMessage("FAULT", "NO FLOW");
191       faultLatched = true;
192       refillSessionActive = false;
193       faultState();
194       server.send(200, "text/plain", "FAULT");
195       return;
196     }
197
198     server.send(200, "text/plain", "OPEN");
199   }
200
201   static void handleStatus()
202   {
203     float waterLevel = getWaterLevelCM();
204     bool chemicalOk = digitalRead(CHEMICAL_OK_PIN) == LOW;
205
```

```
206   String payload = "water=" + String(waterLevel, 2) + " chemical=" +
          String(chemicalOk ? 1 : 0) + " pulses=" + String(flowPulseCount)
          + " active=" + String(refillSessionActive ? 1 : 0);
207   server.send(200, "text/plain", payload);
208 }
209
210 static void handleHeartbeat()
211 {
212   Serial.println("[HEARTBEAT] received from local node");
213   lastHeartbeatTime = millis();
214   server.send(200, "text/plain", "OK");
215 }
216
217 static void handleAnnounce()
218 {
219   String ip = server.arg("ip");
220   if (ip.length() == 0) ip = "unknown";
221   Serial.print("[ANNOUNCE] Local node online, IP=");
222   Serial.println(ip);
223   lastHeartbeatTime = millis();
224   server.send(200, "text/plain", "OK");
225 }
226
227 static void handleDebug()
228 {
229   String msg = server.arg("msg");
230   if (msg.length() > 0) {
231     Serial.print("[LOCAL_NODE] ");
232     Serial.println(msg);
233   }
234   server.send(200, "text/plain", "OK");
235 }
236
237 static void setupWiFiStationAndServer()
238 {
239   WiFi.mode(WIFI_STA);
240   WiFi.persistent(false);
241   WiFi.setAutoReconnect(true);
242   Serial.print("[WIFI] Connecting to ");
243   Serial.println(AP_SSID);
244   // Connect to Wokwi open AP
245   WiFi.begin(AP_SSID);
246
247   unsigned long start = millis();
248   const unsigned long timeout = 5000;
249   while (WiFi.status() != WL_CONNECTED && millis() - start < timeout)
250   {
251     delay(100);
252   }
253
254   if (WiFi.status() == WL_CONNECTED)
255   {
256     Serial.print("[WIFI] Connected, IP=");
257     Serial.println(WiFi.localIP());
258     // advertise mDNS name so other boards can reach us via
          central.local
259     if (MDNS.begin("central"))
260     {
```

```
261       Serial.println("[MDNS] responder started: central.local");
262     }
263   }
264   else
265   {
266     Serial.println("[WIFI] WARNING: failed to join AP, server will
          still run but may not be reachable by other nodes");
267   }
268
269   server.on("/request", HTTP_GET, handleRequest);
270   server.on("/control", HTTP_GET, handleControl);
271   server.on("/status", HTTP_GET, handleStatus);
272   server.on("/heartbeat", HTTP_GET, handleHeartbeat);
273   server.on("/announce", HTTP_GET, handleAnnounce);
274   server.on("/debug", HTTP_GET, handleDebug);
275   server.begin();
276   Serial.println("[HTTP] server started");
277 }
278
279 static void handleSerialCommands()
280 {
281   if (Serial.available() == 0)
282   {
283     return;
284   }
285
286   String cmd = Serial.readStringUntil('\n');
287   cmd.trim();
288
289   if (cmd == "R" || cmd == "r")
290   {
291     simulatedDistanceCM = -1.0;
292     Serial.println("[SIM] distance override disabled");
293   }
294   else if (cmd.startsWith("D") || cmd.startsWith("d"))
295   {
296     simulatedDistanceCM = cmd.substring(1).toFloat();
297     Serial.print("[SIM] distance set to ");
298     Serial.println(simulatedDistanceCM);
299   }
300   else if (cmd == "ANODE")
301   {
302     rgbCommonAnode = true;
303     Serial.println("[CFG] RGB common anode");
304   }
305   else if (cmd == "CATHODE")
306   {
307     rgbCommonAnode = false;
308     Serial.println("[CFG] RGB common cathode");
309   }
310 }
311
312 static void monitorWaterLevel()
313 {
314   float waterLevel = getWaterLevelCM();
315   if (waterLevel < LOW_WATER_THRESHOLD_CM)
316   {
317     if (!lowWaterAlertActive)
```

```
318       {
319         lowWaterAlertActive = true;
320         faultLatched = true;
321         refillSessionActive = false;
322         displayMessage("FAULT", "LOW WATER");
323         faultState();
324       }
325       return;
326     }
327
328     if (lowWaterAlertActive)
329     {
330       lowWaterAlertActive = false;
331       faultLatched = false;
332     }
333
334     if (!refillSessionActive && !faultLatched)
335     {
336       setRGBColor(false, true, false);
337       displayMessage("Water: " + String(waterLevel, 1) + " cm", "System
            Ready");
338     }
339   }
340
341   static void manageRefill()
342   {
343     if (!refillSessionActive)
344     {
345       return;
346     }
347
348     if (flowPulseCount >= TARGET_PULSES)
349     {
350       return;
351     }
352
353     if (flowPulseCount == 0)
354     {
355       setRGBColor(true, true, false);
356     }
357     else
358     {
359       setRGBColor(false, false, true);
360     }
361
362     display.clearDisplay();
363     display.setCursor(0, 0);
364     display.println("REFILL ACTIVE");
365     display.print("Pulses: ");
366     display.print(flowPulseCount);
367     display.print("/");
368     display.println(TARGET_PULSES);
369     display.display();
370   }
371
372   void setup()
373   {
374     Serial.begin(115200);
```

```
  pinMode(FLOW_PULSE_PIN, INPUT_PULLUP);
  pinMode(CHEMICAL_OK_PIN, INPUT_PULLUP);
  pinMode(RGB_RED_PIN, OUTPUT);
  pinMode(RGB_GREEN_PIN, OUTPUT);
  pinMode(RGB_BLUE_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  attachInterrupt(digitalPinToInterrupt(FLOW_PULSE_PIN), flowISR,
      FALLING);

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
  {
    while (true)
    {
      delay(10);
    }
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  setRGBColor(false, true, false);
  setupWiFiStationAndServer();
  displayMessage("System Started", "Ready");
}


void loop()
{
  server.handleClient();
  handleSerialCommands();
  monitorWaterLevel();
  manageRefill();
  // heartbeat visual indicator: briefly flash red on controller RGB
      when heartbeat received
  if (millis() - lastHeartbeatTime < HEARTBEAT_LED_MS) {
    setRGBColor(true, false, false);
  }
  delay(50);
}
```

Listing 8.2: Central controller source code

8.2.3 Local Node Source Code

```
#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <ESP32Servo.h>

// Use Wokwi default virtual WiFi (open, no password)
const char *AP_SSID = "Wokwi-GUEST";
const char *AP_PASSWORD = ""; // empty for open network
const char *CENTRAL_URL = "http://central.local";

constexpr unsigned long WIFI_RETRY_INTERVAL_MS = 3000;
```

```
12 const unsigned long WIFI_CONNECT_TIMEOUT_MS = 10000;
13 const unsigned long DISCOVERY_RETRY_INTERVAL_MS = 2000;
14
15 #define TANK_REQUEST_PIN 13
16 #define SERVO_PIN 4
17 #define BLINK_PIN 26
18 #define HEARTBEAT_BTN_PIN 15
19
20 Servo valveServo;
21
22 bool refillActive = false;
23 bool pendingRefillRequest = false;
24 bool heartbeatEnabled = false;
25 bool pendingHeartbeatRequest = false;
26 unsigned long lastControlPoll = 0;
27 unsigned long lastButtonEventTime = 0;
28 int lastButtonState = HIGH;
29 unsigned long lastWiFiAttempt = 0;
30 bool wifiConnectedLogged = false;
31 unsigned long wifiConnectStartTime = 0;
32 unsigned long lastDiscoveryAttempt = 0;
33 bool centralDiscovered = false;
34
35 // Blink state (non-blocking)
36 unsigned long lastBlinkMillis = 0;
37 bool blinkState = false;
38 const unsigned long BLINK_INTERVAL_MS = 500;
39
40 // Heartbeat pattern when connected: two short blinks every
       HEARTBEAT_PERIOD_MS
41 const unsigned long HEARTBEAT_SHORT_MS = 80;
42 const unsigned long HEARTBEAT_GAP_MS = 100;
43 const unsigned long HEARTBEAT_PERIOD_MS = 1500;
44
45 static void setValveOpen(bool open)
46 {
47   valveServo.write(open ? 90 : 0);
48 }
49
50 static String httpGet(const String &path)
51 {
52   if (WiFi.status() != WL_CONNECTED)
53   {
54     Serial.println("[HTTP] WiFi not connected, skipping");
55     return String();
56   }
57
58   String url = String(CENTRAL_URL) + path;
59   Serial.print("[HTTP] GET ");
60   Serial.println(url);
61
62   WiFiClient client;
63   HTTPClient http;
64   if (!http.begin(client, url))
65   {
66     Serial.println("[HTTP] begin() failed");
67     return String();
68   }
```

```
69    int code = http.GET();
70    String payload = (code > 0) ? http.getString() : String();
71    Serial.print("[HTTP] response code=");
72    Serial.println(code);
73    http.end();
74    return payload;
75  }
76
77  static void connectToCentralWiFi()
78  {
79    if (WiFi.status() == WL_CONNECTED)
80    {
81      return;
82    }
83    WiFi.mode(WIFI_STA);
84    WiFi.persistent(false);
85    WiFi.setAutoReconnect(true);
86    WiFi.setSleep(false);
87    Serial.print("[WIFI] Connecting to ");
88    Serial.println(AP_SSID);
89    if (AP_PASSWORD[0] == '\0')
90    {
91      WiFi.begin(AP_SSID);
92    }
93    else
94    {
95      WiFi.begin(AP_SSID, AP_PASSWORD);
96    }
97    wifiConnectStartTime = millis();
98  }
99
100 static void maintainCentralWiFi()
101 {
102   wl_status_t status = WiFi.status();
103
104   if (status == WL_CONNECTED)
105   {
106     if (!wifiConnectedLogged)
107     {
108       wifiConnectedLogged = true;
109       Serial.print("[WIFI] Connected, IP=");
110       Serial.println(WiFi.localIP());
111       lastDiscoveryAttempt = 0;
112     }
113     return;
114   }
115
116   wifiConnectedLogged = false;
117
118   if (wifiConnectStartTime > 0 && millis() - wifiConnectStartTime >
        WIFI_CONNECT_TIMEOUT_MS)
119   {
120     Serial.println("[WIFI] Connection timeout, resetting...");
121     WiFi.disconnect();
122     wifiConnectStartTime = 0;
123     lastWiFiAttempt = millis();
124     return;
125   }
```

```
126
127   if (millis() - lastWiFiAttempt >= WIFI_RETRY_INTERVAL_MS)
128   {
129     lastWiFiAttempt = millis();
130     Serial.print("[WIFI] Status: ");
131     Serial.println((int)status);
132     connectToCentralWiFi();
133   }
134 }
135
136 static void requestRefill()
137 {
138   Serial.println("[NODE] Sending refill request to central
        controller");
139   String response = httpGet("/request");
140   if (response.length() == 0)
141   {
142     Serial.println("[NODE] Central request failed, will retry");
143     refillActive = false;
144     setValveOpen(false);
145     return;
146   }
147
148   Serial.print("[NODE] Central response: ");
149   Serial.println(response);
150   if (response.indexOf("APPROVE") >= 0)
151   {
152     refillActive = true;
153     pendingRefillRequest = false;
154     setValveOpen(true);
155   }
156   else
157   {
158     refillActive = false;
159     pendingRefillRequest = false;
160     setValveOpen(false);
161   }
162 }
163
164 static void processPendingRequest()
165 {
166   if (!pendingRefillRequest)
167   {
168     return;
169   }
170
171   if (WiFi.status() != WL_CONNECTED)
172   {
173     return;
174   }
175
176   requestRefill();
177 }
178
179 static void pollControl()
180 {
181   if (!refillActive)
182   {
```

```
    return;
  }
  if (millis() - lastControlPoll < 250)
  {
    return;
  }
  lastControlPoll = millis();
  String response = httpGet("/control");
  if (response.length() == 0)
  {
    return;
  }
  if (response.indexOf("OPEN") >= 0)
  {
    setValveOpen(true);
  }
  else if (response.indexOf("CLOSE") >= 0)
  {
    setValveOpen(false);
    refillActive = false;
  }
  else if (response.indexOf("FAULT") >= 0 || response.indexOf("DENY")
      >= 0)
  {
    setValveOpen(false);
    refillActive = false;
  }
}

void setup()
{
  Serial.begin(115200);
  pinMode(TANK_REQUEST_PIN, INPUT_PULLUP);
  pinMode(HEARTBEAT_BTN_PIN, INPUT_PULLUP);
  valveServo.attach(SERVO_PIN);
  setValveOpen(false);
  pinMode(BLINK_PIN, OUTPUT);
  digitalWrite(BLINK_PIN, LOW);
  lastWiFiAttempt = millis();
  connectToCentralWiFi();
}

void loop()
{
  maintainCentralWiFi();
  // LED heartbeat: fast heartbeat when WiFi connected, slow blink
      when disconnected
  unsigned long now = millis();
  bool wifiReady = (WiFi.status() == WL_CONNECTED);

  if (wifiReady)
  {
    if (heartbeatEnabled)
    {
      unsigned long t = now % HEARTBEAT_PERIOD_MS;
      if (t < HEARTBEAT_SHORT_MS)
      {
        digitalWrite(BLINK_PIN, HIGH);
```

```
239       }
240       else if (t < HEARTBEAT_SHORT_MS + HEARTBEAT_GAP_MS)
241       {
242         digitalWrite(BLINK_PIN, LOW);
243       }
244       else if (t < HEARTBEAT_SHORT_MS + HEARTBEAT_GAP_MS +
              HEARTBEAT_SHORT_MS)
245       {
246         digitalWrite(BLINK_PIN, HIGH);
247       }
248       else
249       {
250         digitalWrite(BLINK_PIN, LOW);
251       }
252       // send a lightweight heartbeat to central once per heartbeat
              period
253       static unsigned long lastHeartbeatSent = 0;
254       if (now - lastHeartbeatSent >= HEARTBEAT_PERIOD_MS)
255       {
256         lastHeartbeatSent = now;
257         (void)httpGet("/heartbeat");
258       }
259     }
260     else
261     {
262       // WiFi connected but heartbeat not enabled: keep LED off
263       digitalWrite(BLINK_PIN, LOW);
264     }
265   }
266   else
267   {
268     if (now - lastBlinkMillis >= BLINK_INTERVAL_MS)
269     {
270       lastBlinkMillis = now;
271       blinkState = !blinkState;
272       digitalWrite(BLINK_PIN, blinkState ? HIGH : LOW);
273       Serial.print("[BLINK] pin ");
274       Serial.print(BLINK_PIN);
275       Serial.print(" -> ");
276       Serial.println(blinkState ? "HIGH" : "LOW");
277     }
278   }
279
280   // Handle tank request button (existing behavior)
281   int buttonState = digitalRead(TANK_REQUEST_PIN);
282   if (buttonState != lastButtonState)
283   {
284     unsigned long nowBtn = millis();
285     if (nowBtn - lastButtonEventTime > 250)
286     {
287       delay(20);
288       int stable = digitalRead(TANK_REQUEST_PIN);
289       if (stable != lastButtonState && stable == LOW)
290       {
291         lastButtonEventTime = nowBtn;
292         pendingRefillRequest = true;
293         requestRefill();
294       }
```

```
295         lastButtonState = stable;
296       }
297     }
298
299     // Handle heartbeat enable button: press to request heartbeat
        activation
300     static int lastHbBtnState = HIGH;
301     int hbState = digitalRead(HEARTBEAT_BTN_PIN);
302     if (hbState != lastHbBtnState)
303     {
304       unsigned long nowHb = millis();
305       if (nowHb - lastButtonEventTime > 250)
306       {
307         delay(20);
308         int stable = digitalRead(HEARTBEAT_BTN_PIN);
309         if (stable != lastHbBtnState && stable == LOW)
310         {
311           // user pressed heartbeat enable
312           pendingHeartbeatRequest = true;
313           Serial.println("[HEARTBEAT] enable button pressed");
314         }
315         lastHbBtnState = stable;
316       }
317     }
318
319     processPendingRequest();
320
321     // Attempt discovery of central controller
322     if (WiFi.status() == WL_CONNECTED && millis() - lastDiscoveryAttempt
        >= DISCOVERY_RETRY_INTERVAL_MS)
323     {
324       lastDiscoveryAttempt = millis();
325       if (!centralDiscovered)
326       {
327         Serial.println("[DISCOVERY] Pinging central...");
328         String response = httpGet("/announce?ip=" +
              WiFi.localIP().toString());
329         if (response.length() > 0)
330         {
331           centralDiscovered = true;
332           Serial.println("[DISCOVERY] Central reachable!");
333         }
334       }
335     }
336
337     // process heartbeat pending request: enable when WiFi connected
338     if (pendingHeartbeatRequest)
339     {
340       if (WiFi.status() == WL_CONNECTED)
341       {
342         heartbeatEnabled = true;
343         pendingHeartbeatRequest = false;
344         Serial.println("[HEARTBEAT] enabled (WiFi connected)");
345       }
346     }
347     pollControl();
348     delay(50);
349 }
```

Listing 8.3: Local node source code

Conclusion

An IoT-based automated hospital chemical dispensing system was designed and simulated. It uses an ESP32 controller with ultrasonic, load cell, flow, and capacitive sensors, plus peristaltic and diaphragm pumps for accurate dosing and mixing. Multiple safety features detect leaks, overflows, and sensor faults. WiFi with MQTT enables real-time monitoring and alerts, while the cloud platform is used for monitoring and logging rather than direct control. The Wokwi simulation validated the core logic for one dispensing node in the larger distributed architecture.

References

[1] Wokwi, *Online ESP32 Simulator and Firmware Builder*. [Online]. Available: https://wokwi.com/

[2] Wikipedia, *ESP32*. [Online]. Available: https://en.wikipedia.org/wiki/ESP32

[3] YouTube, "Connect ESP32 to WiFi using Visual Studio Code and PlatformIO (Beginner Tutorial)", 2024. [Online]. Available: https://youtu.be/aAG0bp0Q-y4?si=hmnsah4a_ZVhALW4.

[4] Vseeds.lk, "ESP32 DevKit V1 Development Board," 2025. [Online]. Available: https://www.vseeds.lk/products/esp32-dev-kit-v1

[5] Tronic.lk, "JSN-SR04T Waterproof Ultrasonic Distance Measuring Module," 2025. [Online]. Available: https://tronic.lk/product/jsn-sr04t-waterproof-ultrasonic-distance-measuring-modu

[6] Tronic.lk, "SSD1306 OLED Display Module," 2025. [Online]. Available: https://tronic.lk/

[7] Daraz.lk, "RGB LED Module," 2025. [Online]. Available: https://www.daraz.lk/

[8] YouTube, "Zigbee Overview," 2024. [Online]. Available: https://youtu.be/lVr88JS-2rM?si=Po6g7NyB4-zrrfi-

[9] Wikipedia, *LoRa*. [Online]. Available: https://en.wikipedia.org/wiki/LoRa

[10] OpenAI, "ChatGPT," used to help generate the system architecture image. [Online]. Available: https://chat.openai.com/.